



CMPT 295

Unit - Machine-Level Programming

Lecture 21 – Assembly language –

Function Call and Stack – Recursion – Annotated!

M []

Stack

Bottom

Stack frame
of 1st
invocation
of *func*

Stack frame
of 2nd
invocation
of *func*

Stack frame
of 3rd
invocation
of *func*

Top

Recursion in x86-64

We already know
how to implement
recursive functions
in x86-64 assembly
language!

- Handled without special instruction

1. Call / return mechanism still follows the *stack pattern*

➤ If *func* (first invocation of *func*) calls itself (second invocation of *func*), then the second invocation of *func* must terminate before the first invocation of *func* can do so

2. Each invocation of the recursive function has its own *stack frame*

- Saved registers & local variables

- Argument 7.. n (if any)

- Saved return address

- *Function call conventions* still required and implemented

- Passing *arguments* and returning *return value*

- *Register saving conventions* still required and implemented

- Prevent one function invocation from corrupting another's data (stored in registers)

- Also works for mutual (or indirect) recursion

- *func1* calls *func2* then *func2* calls *func1*

Recursive Function – countOnesR (...)

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

What does this function do?

Recursive Function – Example – Base Case

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

```
countOnesR:  
    xorl    %eax, %eax  
    testq   %rdi, %rdi  
    je      done  
    pushq   %rbx  
    movq    %rdi, %rbx  
    andl    $1, %ebx  
    shrq    %rdi  
    call    countOnesR  
    addq    %rbx, %rax  
    popq    %rbx  
done:  
    ret
```

Recursive Function – Example – Callee saved registers

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

matched

```
countOnesR:  
    xorl    %eax, %eax  
    testq   %rdi, %rdi  
    je      done  
    pushq   %rbx  
    movq    %rdi, %rbx  
    andl    $1, %ebx  
    shrq    %rdi  
    call    countOnesR  
    addq    %rbx, %rax  
    popq    %rbx  
done:  
    ret
```

Recursive Function – Example – Arguments setup

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

```
countOnesR:  
    xorl    %eax, %eax  
    testq   %rdi, %rdi  
    je      done  
    pushq   %rbx  
    movq    %rdi, %rbx  
    mask andl    $1, %ebx  
    shrq    %rdi implicit operand: 1  
    call    countOnesR  
    addq    %rbx, %rax  
    popq    %rbx  
done:  
    ret
```

%rdi: x = 5 -> 00...0101

& -> 00...0001 *mask*

00...0001

and'ing an operand (here: 7) with a mask (here: 1)
allows us to select/isolate specific bit(s) in the
operand by setting the matching bit(s) in the mask.

Recursive Function – Example – Recursive Call

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

return from call
here!

```
countOnesR:  
    xorl    %eax, %eax  
    testq   %rdi, %rdi  
    je      done  
    pushq   %rbx  
    movq    %rdi, %rbx  
    andl    $1, %ebx  
    shrq    %rdi  
    call    countOnesR  
    addq    %rbx, %rax  
    popq    %rbx  
done:  
    ret
```

Recursive Function – Example – Result

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

(i.e., %rbx)

```
countOnesR:  
    xorl    %eax, %eax  
    testq   %rdi, %rdi  
    je      done  
    pushq   %rbx  
    movq    %rdi, %rbx  
    andl    $1, %ebx  
    shrq    %rdi  
    call    countOnesR  
    addq    %rbx, %rax  
    popq    %rbx  
done:  
    ret
```

Recursive Function – Example – Restore stack and return

```
/* Recursive counter of 1's */
long countOnesR(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1) + countOnesR(x >> 1);
}
```

```
countOnesR:
    xorl    %eax, %eax
    testq   %rdi, %rdi
    je      done
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    countOnesR
    addq    %rbx, %rax
    popq    %rbx
done:
    ret
```

Recursive Function – Example – Test Cases

- ▶ Test Case 1
 - ▶ Input: $x = 0$
 - ▶ Expected result: 0
- ▶ Test Case 2
 - ▶ Input: $x = 7$
 - ▶ Expected result: 3

$$\frac{\text{base} + \text{displacement}}{2}$$

Stack Variables

For you to practice
drawing the stack

Register Table :

[illegible]

countOnesR:

```
xorl    %eax, %eax
```

```
testq    %rdi, %rdi
```

je done

pushq %rbx

```
movq    %rdi, %rbx
```

```
andl    $1, %ebx
```

shrq %rdi

call **countOnesR**

```
addq    %rbx, %rax
```

popq %rbx

done :

ret